

Cache Design Exploration

Direct-Mapped vs 2-Way vs 4-Way Set Associative Cache

Noah Dumas

CS 4200: Computer Architecture

Instructor: Dr. Zanyar ZohourianShahzadi

University of Colorado Colorado Springs

May 4th 2026

Project Goal

The goal of this project is to compare different cache organizations and measure how cache structure affects memory performance.

- Direct-mapped cache
- 2-way set associative cache
- 4-way set associative cache
- Same cache size and block size for fair comparison

Why Caches Matter

Caches are small, fast memories placed close to the CPU.

They improve performance by exploiting:

- **Temporal locality:** recently used data is likely to be used again
- **Spatial locality:** nearby data is likely to be accessed soon

A higher hit rate means fewer slow memory accesses.

Cache Configurations

Cache Type	Size	Block Size	Associativity
Direct-Mapped	256 B	16 B	1-way
2-Way Set Associative	256 B	16 B	2-way
4-Way Set Associative	256 B	16 B	4-way

All configurations use write-back, write-allocate, and LRU replacement where applicable.

Address Breakdown

Each memory address is split into three parts:

$$\text{Address} = \text{Tag} \mid \text{Index} \mid \text{Block Offset}$$

- **Tag:** identifies which memory block is stored
- **Index:** selects the cache set
- **Offset:** selects the byte inside the block

The simulator processes memory operations from a trace file.

Each access performs:

- ① Decode address into tag, index, and offset
- ② Search selected cache set
- ③ Record hit or miss
- ④ Replace a line using LRU if needed
- ⑤ Track dirty writebacks on eviction

Code Structure

The simulator uses two main data structures:

```
@dataclass
class CacheLine:
    valid: bool = False
    dirty: bool = False
    tag: int = 0
    last_used: int = 0

@dataclass
class CacheStats:
    accesses: int = 0
    hits: int = 0
    misses: int = 0
    evictions: int = 0
    dirty_writebacks: int = 0
```

CacheLine stores one cache entry. CacheStats tracks hits, misses, evictions, and writebacks.

Important Code: Address Decoding

The address is decoded into tag, index, offset, and block number.

```
def decode_address(self, address):  
    block_number = address // self.block_size_bytes  
    block_offset = address % self.block_size_bytes  
    set_index = block_number % self.num_sets  
    tag = block_number // self.num_sets  
  
    return tag, set_index, block_offset, block_number
```

This is the part that connects the program to the cache mapping rules.

Important Code: Hit, Miss, and LRU

```
for way, line in enumerate(cache_set):
    if line.valid and line.tag == tag:
        self.stats.hits += 1
        line.last_used = self.clock

        if operation == "S":
            line.dirty = True

        return "HIT"

self.stats.misses += 1
victim_way = self._choose_victim_way(cache_set
)
```

The simulator first searches for a matching valid tag. On a miss, it selects the least recently used line as the replacement victim.

For the demo, I will run the simulator and show the generated output files.

- Run: `python cache_design_exploration.py`
- Show `results.log` for the summary table
- Show `results.csv` for experiment data
- Show `trace.log` for hit, miss, eviction, and writeback details

The results show that higher associativity can reduce conflict misses.

- Sequential and repeated-loop traces performed well on all cache types
- Conflict-heavy traces exposed the weakness of direct-mapped caches
- The 4-way cache handled conflicts better because each set had more available lines
- Fewer evictions also reduced dirty writebacks in the mixed workload

Experiment Results: Conflict Pattern

Cache Type	Hits	Misses	Hit Rate	Evictions
Direct-Mapped	0	32	0.00%	31
2-Way Set Associative	0	32	0.00%	30
4-Way Set Associative	28	4	87.50%	0

The 4-way cache performed much better because it could hold all four conflicting blocks in the same set.

Experiment Results: Mixed Workload

Cache Type	Hits	Misses	Hit Rate	Evictions	Writebacks
Direct-Mapped	6	10	37.50%	8	5
2-Way Set Associative	6	10	37.50%	7	5
4-Way Set Associative	10	6	62.50%	1	1

The 4-way cache had fewer misses, fewer evictions, and fewer dirty writebacks.

This project shows that cache organization has a major effect on memory performance.

- Direct-mapped caches are simple, but they suffer from conflict misses
- 2-way caches can help, but still fail on some access patterns
- 4-way set associative caches gave the best results in the conflict and mixed traces
- The tradeoff is that higher associativity needs more hardware comparison logic